

计算机系统基础 (CSAPP) 综合模拟试卷

命题人: CSemy

审题人: Rikka

2025.12.26

考试说明: 本试卷共五大题, 满分100分。请仔细阅读题目要求, 在答题纸上作答。

1 单项选择题 (每题 2 分, 共 20 分)

1. 在 64 位 Linux 系统 (x86-64) 中, 以下 C 语言数据类型所占字节数描述**错误**的是:

- A. unsigned long 占 8 字节
- B. char * 占 8 字节
- C. int 占 4 字节
- D. float 占 8 字节

2. 假设变量 `x` 的类型为 `int`, 其机器数表示为 `0x80000000`。则表达式 `(x >> 31)` 在算术右移下的结果是:

- A. `0x00000001`
- B. `0xFFFFFFFF`
- C. `0x00000000`
- D. `0x80000000`

3. 关于 IEEE 754 单精度浮点数 (32位), 下列说法正确的是:

- A. 阶码 (Exponent) 部分采用补码表示。
- B. 尾数 (Significand) 部分隐含了一个开头的 1, 除非是去规格化数 (Denormalized)。
- C. `+0.0` 和 `-0.0` 的二进制位模式完全相同。
- D. 浮点数加法满足结合律, 即 $(a+b)+c$ 总是等于 $a+(b+c)$ 。

4. 在 C 语言程序到可执行文件的转换过程中, 负责处理以 `#` 开头的指令 (如 `#include`, `#define`) 的阶段是:

- A. 预处理 (Preprocessing)
- B. 编译 (Compilation)
- C. 汇编 (Assembly)

D. 链接 (Linking)

5. 假设 `%rax` 寄存器的值为 `0x100`, `%rcx` 寄存器的值为 `0x2`。执行指令 `leaq 8(%rax, %rcx, 4), %rdx` 后, `%rdx` 的值是:

- A. `0x10A`
- B. `0x108`
- C. `0x408`
- D. `0x110`

6. 在 x86-64 过程调用中, 以下属于**被调用者保存** (Callee-saved) 寄存器的是:

- A. `%rax`
- B. `%rdi`
- C. `%rbx`
- D. `%rcx`

7. 关于 ELF 目标文件中的符号表 (`.symtab`), 下列说法**错误**的是:

- A. 包含全局变量和函数的定义与引用信息。
- B. 局部静态变量 (`static` 修饰的局部变量) 也会出现在符号表中。
- C. 所有的局部变量 (函数内部非 `static` 变量) 都保存在符号表中。
- D. 强符号 (Strong symbol) 通常指的是已初始化的全局变量或函数。

8. 下列关于缓冲区溢出 (Buffer Overflow) 的说法中, 正确的是:

- A. 只要使用 `fgets` 就可以完全避免缓冲区溢出攻击。
- B. 栈随机化 (ASLR) 通过每次运行程序时改变栈的位置来防止攻击者预测地址。
- C. 金丝雀 (Canary) 值通常保存在栈帧的返回地址上方 (即高地址处)。
- D. 缓冲区溢出只会导致程序崩溃, 不会造成安全漏洞。

9. 在 Unix I/O 中, 执行 `dup2(oldfd, newfd)` 函数的作用是:

- A. 创建一个新的文件描述符, 其数值为 `oldfd + 1`。
- B. 关闭 `oldfd`, 并将 `newfd` 指向 `oldfd` 原来的文件表项。
- C. 如果 `newfd` 已经打开, 先关闭它, 然后让 `newfd` 指向 `oldfd` 指向的文件表项。
- D. 将 `newfd` 的内容复制到 `oldfd` 中。

10. 父进程调用 `fork()` 创建子进程时, 下列哪项资源**不是**父子进程共享或副本继承的?

- A. 打开的文件描述符表
- B. 虚拟地址空间 (代码段、数据段、堆、栈) 的内容
- C. 进程 ID (PID)
- D. 程序计数器 (PC) 的值

2 数据表示与运算 (共 15 分)

1. (5分) 整数与位运算
假设字长为 8 位（仅用于本题），请补全下表。

十进制值	二进制补码表示	无符号数值	对应十六进制
-25		N/A	
	1010 1100		0xAC
TMin (8位)			

2. (5分) 浮点数编码
考虑一个 5 位的浮点数表示，格式为 1 位符号位，2 位阶码（偏置量 Bias 为 $2^{2-1} - 1 = 1$ ），2 位尾数。请填写下表：

位模式 (S E M)	类型(规格/非规格)	阶码真实值(E)	尾数真实值(M)	十进制数值(V)
0 10 01	规格化			
0 00 10				

3. (5分) C语言类型转换与截断
假设在 32 位机器上运行以下代码，请写出输出结果。

```
1 int x = -1;
2 unsigned u = 2147483648; // 2^31
3 printf("x > u = %d\n", x > u);
4
5 short s = -12345;
6 unsigned short us = s;
7 int y = us;
8 printf("y = %d\n", y);
```

3 机器级编程与栈帧分析 (共 30 分)

1. (15分) 汇编代码还原
给定如下 C 语言函数 loop_func 和对应的 x86-64 汇编代码。请根据汇编逻辑补全 C 代码中的空缺。

汇编代码:

```
1 ; long loop_func(long x, long y)
2 ; x in %rdi, y in %rsi
3 loop_func:
4     movq    %rdi, %rax
5     movq    $0, %rcx
6 .L2:
7     cmpq    %rsi, %rcx
8     jge     .L4
9     leaq    (%rax,%rax,2), %rax
10    addq    %rcx, %rax
11    addq    $1, %rcx
12    jmp     .L2
13 .L4:
14    ret
```

C 语言代码:

```
1 long loop_func(long x, long y) {
2     long result = x;
3     long i;
4     for (i = 0; _____(1)_____; i++) {
5
6         result = _____(2)_____;
7
8         result += i;
9     }
10    return result;
11 }
```

问题:

1. 填空 (1): _____

2. 填空 (2): _____ (请用关于 result 的表达式表示)

3. 如果调用 loop_func(2, 3), 函数的返回值是多少? _____

2. (15分) 结构体与对齐

考虑以下结构体定义 (x86-64 Linux 环境, 对齐系数为默认值):

```
1 struct Node {
2     char c;
3     int i;
4     char s[3];
5     double d;
6     struct Node *next;
7 };
```

1. 无论成员顺序如何, double 类型在结构体内部的偏移量 (Offset) 必须是 _____ 的倍数。
2. 请画出该结构体的内存布局, 标出每个成员的偏移量 (Offset) 和填充 (Padding) 的位置及大小。
3. 该结构体 struct Node 的总大小 (sizeof) 是多少字节?
4. 如果重新排列成员顺序以最小化内存占用, 最小的大小是多少字节? 请给出一种排序方案。

4 链接与异常控制流 (共 20 分)

1. (10分) 符号解析

假设有两个 C 文件 main.c 和 lib.c, 它们被编译并链接在一起。

<pre>/* main.c */ 1 #include <stdio.h> 2 void foo(void); 3 4 int x = 100; 5 int y = 200; 6 7 int main() { 8 foo(); 9 printf("x=%d, y=%d\n", x, y); 10 return 0; 11 }</pre>	<pre>/* lib.c */ 1 double x; 2 void foo() { 3 x = -0.0; 4 }</pre>
---	---

问题:

1. 在链接过程中, main.c 中的 int x = 100 是强符号还是弱符号? lib.c 中的 double x 是强符号还是弱符号?
2. 链接器会报错吗? 如果不会, 链接器会选择哪个 x 的定义?
3. 在 x86-64 系统 (小端) 中, 执行完 foo() 后, main 函数中打印的 x 和 y 的值分别是多少? (提示: -0.0 的十六进制表示为 0x8000000000000000)。

2. (10分) 进程控制

阅读以下代码, 写出其可能的输出序列。如果有多组可能, 写出其中两组。

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <sys/wait.h>
5
6 int main() {
7     if (fork() == 0) {
8         printf("A");
9         fflush(stdout);
10    } else {
11        printf("B");
12        fflush(stdout);
13        wait(NULL);
14        printf("C");
15        fflush(stdout);
16    }
17    printf("D");
18    fflush(stdout);
19    exit(0);
20 }
```

提示： 注意 `exit(0)` 会终止当前进程。

5 I/O 系统 (共 15 分)

1. (5分) Unix I/O 中，标准输入、标准输出、标准错误的默认文件描述符分别是 _____、_____ 和 _____。

2. (10分) I/O 重定向

假设文件 `data.txt` 的内容为 "12345"。阅读以下代码片段：

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4
5 int main() {
6     int fd1, fd2;
7     char c;
8     fd1 = open("data.txt", O_RDONLY, 0);
9     fd2 = open("data.txt", O_RDONLY, 0);
10    read(fd1, &c, 1);
11    read(fd2, &c, 1);
12    printf("c = %c\n", c);
13    close(fd2);
14    dup2(fd1, fd2);
15    read(fd2, &c, 1);
16    printf("c = %c\n", c);
17    close(fd1);
18    return 0;
19 }
```

问题：

1. 两次打开同一个文件，内核会维护几个“打开文件表”表项？
2. 第一条 `printf` 输出什么？
3. 第二条 `printf` 输出什么？请简述原因。
4. 代码中缺少了哪些必要的头文件？（已在代码框内补充，请指出）。

答案与解析

一、单项选择题 (20分)

1. **D**。float 在 x86-64 中仍为 4 字节，double 为 8 字节。
2. **B**。负数算术右移补符号位。
3. **B**。A 错（阶码为偏置量）；C 错（符号位不同）；D 错（浮点数不满足结合律）。
4. **A**。预处理器处理 #。
5. **D**。 $0x100 + 2 * 4 + 8 = 0x110$ 。
6. **C**。rbx, rbp, r12-r15 属于被调用者保存。
7. **C**。非静态局部变量在栈上，不进符号表。
8. **B**。ASLR 通过随机化地址空间增加攻击难度。
9. **C**。这是 dup2 的标准定义。
10. **C**。子进程拥有独立的进程 ID。

二、数据表示与运算 (15分)

1. 整数表：

-25: 11100111 (0xE7)

1010 1100: 有符号为 -84, 无符号 172

TMin: -128, 10000000 (0x80)

2. 浮点数表：

0 10 01: $E = 1, M = 1.25, V = 2.5$

0 00 10: 非规格, $E = 0, M = 0.5, V = 0.5$

3. C语言转换：

-x > u = 1 (混合运算时, -1 转换为最大无符号数)

-y = 53191 (short 负数转 unsigned short 后再零扩展为 int)

三、机器级编程 (30分)

1. 汇编还原：

(1) `i < y`

(2) `result * 3`

(3) **59** (计算过程: $2 \rightarrow 6 \rightarrow 19 \rightarrow 59$)

2. 结构体：

(1) **8**

(3) **32 字节** (布局包含多个 Padding 以对齐)

(4) **24 字节** (比如按照 c,s,i,d,next 的顺序)

四、链接与 ECF (20分)

1. 符号解析：

(1) main 的 x 是强符号，lib 的 x 是弱符号。

(2) 不报错，选强符号。

(3) x=0, y=-2147483648 (0x80000000)。覆盖了 x 和 y 的内存。

2. 进程控制：

序列1: **ABDC D**, 序列2: **BADC D** (关键是 C 必须在 A 和第一个 D 之后)

五、I/O 系统 (15分)

1. **0, 1, 2**。

2. **I/O 重定向：**

- (1) 2 个。
- (2) $c = 1$ 。
- (3) $c = 2$ 。由于 `dup2` 后两者共享文件表项和文件偏移量。